

L4

模型微调与 私有化大模型

全课程知识地图



把一个开源大模型,变成能在自有环境跑起来、跑得好、跑得起的业务系统

第 1 章 认知与基础 开源模型 · 私有化必要性		第 2 章 深度学习与 Transformer 训练循环 · 自注意力		第 3 章 数据工程 格式 · 清洗 · 多样性	
第 4 章 微调技术 LoRA · SWIFT · 评估 M1		第 5 章 模型量化与压缩 GPTQ/AWQ · QLoRA · GGUF M2		第 6 章 推理引擎部署 vLLM · 算容量 · 压测 M3	
第 7 章 RAG 文档处理 解析 · 视觉原生 · 选型		第 8 章 智能体新生态 OpenClaw · MCP · Skill		第 9 章 综合实战 电商客服系统 M4	



智泊AI
ZHIPO AI



魔泊云
MoPaaS

01

CHAPTER

认知与基础

为什么要做私有化大模型?

开源 vs 闭源 · 私有化的四大动因



维度	闭源大模型	开源大模型
代表	GPT-4 · Claude · Gemini	LLaMA · Qwen · DeepSeek
性能	通常领先	差距快速缩小
数据私有	数据需上传 X	可本地部署 ✓
成本	按 Token 计费	硬件 + 电费
定制	仅 Prompt / 部分微调	全量微调自由
可控性	依赖供应商	完全自主

私有化部署的四大动因



数据安全
敏感数据不出域, 合规审计可控



成本可控
高并发场景下边际成本接近 0



深度定制
微调、量化、领域适配自由



合规可审
金融/医疗/政务硬性要求

三大主力开源系列速查



LLaMA

Meta · 2023起

核心特点

- 生态最成熟
- 英文能力强
- MoE 架构
- 支持长上下文

适用场景

英文任务 · 工具链完善

DeepSeek

深度求索 · 中国

核心特点

- 高性能低成本
- 代码 & 数学强
- 推理能力突出
- V2/V3 迭代

适用场景

推理 / 代码 / 中英文

Qwen

阿里巴巴 · 中国

核心特点

- 中文最强
- 覆盖 0.5B–72B
- 多模态(VL/Audio)
- 工具调用原生

适用场景

中文场景 · 课程首选

★ 初学者推荐 Qwen 系列(中文场景效果最稳,本课程综合项目基于 Qwen1.5-1.8B-Chat)

大模型开发全流程一图概览

智泊AI
ZHIPU AI魔泊云
MoPaaS

本课程覆盖范围

★ 微调(第 3-4 章) → 量化(第 5 章) → 部署(第 6 章) → RAG/Agent(第 7-8 章) → 实战(第 9 章)

预训练在本课程不展开,但Transformer 原理在第2 章打基础

模型加载与推理

智泊AI
ZHIPU AI魔泊云
MoPaaS

Transformers

Hugging Face

适用

学习 / 调试 / 离线推理

优势

- ✓ 接口最全
- ✓ 生态最丰富
- ✓ 支持任意模型

局限

- X 性能一般
- X 并发能力弱

```
model = AutoModel
.from_pretrained(
    "Qwen/Qwen2.5-7B"
)
```

vLLM

UC Berkeley + CMU

适用

生产 GPU 服务

优势

- ✓ PagedAttention
- ✓ Continuous Batching
- ✓ OpenAI 兼容 API

局限

- X 仅 GPU
- X 模型支持有限

```
from vllm import LLM
llm = LLM(
    model="qwen2.5-7b"
)
```

Gradio

可视化 Chatbot

适用

Demo / 内部工具

优势

- ✓ 几行代码出 Web UI
- ✓ 支持流式输出
- ✓ 易分享

局限

- X 不适合生产
- X 并发能力弱

```
import gradio as gr
gr.ChatInterface(
    fn=chat_fn
).launch()
```

★ 易错点 & 高频考点



! 第 1 章 复习提示

X 开源 = 免费商用	✓ 看 LICENSE, LLaMA 早期不允许商用; Qwen2/DeepSeek 较宽松, 部分仍有限制
X 参数 B 数 = 文件 GB 数	✓ B = Billion 参数。FP16 下, 1B ≈ 2GB; Qwen2.5-7B 全精度约 14 GB
X 评测榜单分数高 = 业务效果好	✓ 榜单刷分严重, 要看相对位次 + 实际业务测试, 关键看 RAG 一致性与拒答
X 私有部署 = 自动便宜	✓ 低并发场景 API 反而便宜; 只有持续高并发或数据敏感场景私部才划算
X vLLM 万能	✓ vLLM 是通用 GPU 推理事实标准, 但 Agent / 长上下文 SGLang 更优



智泊AI
ZHIPU AI



魔泊云
MoPaaS

02

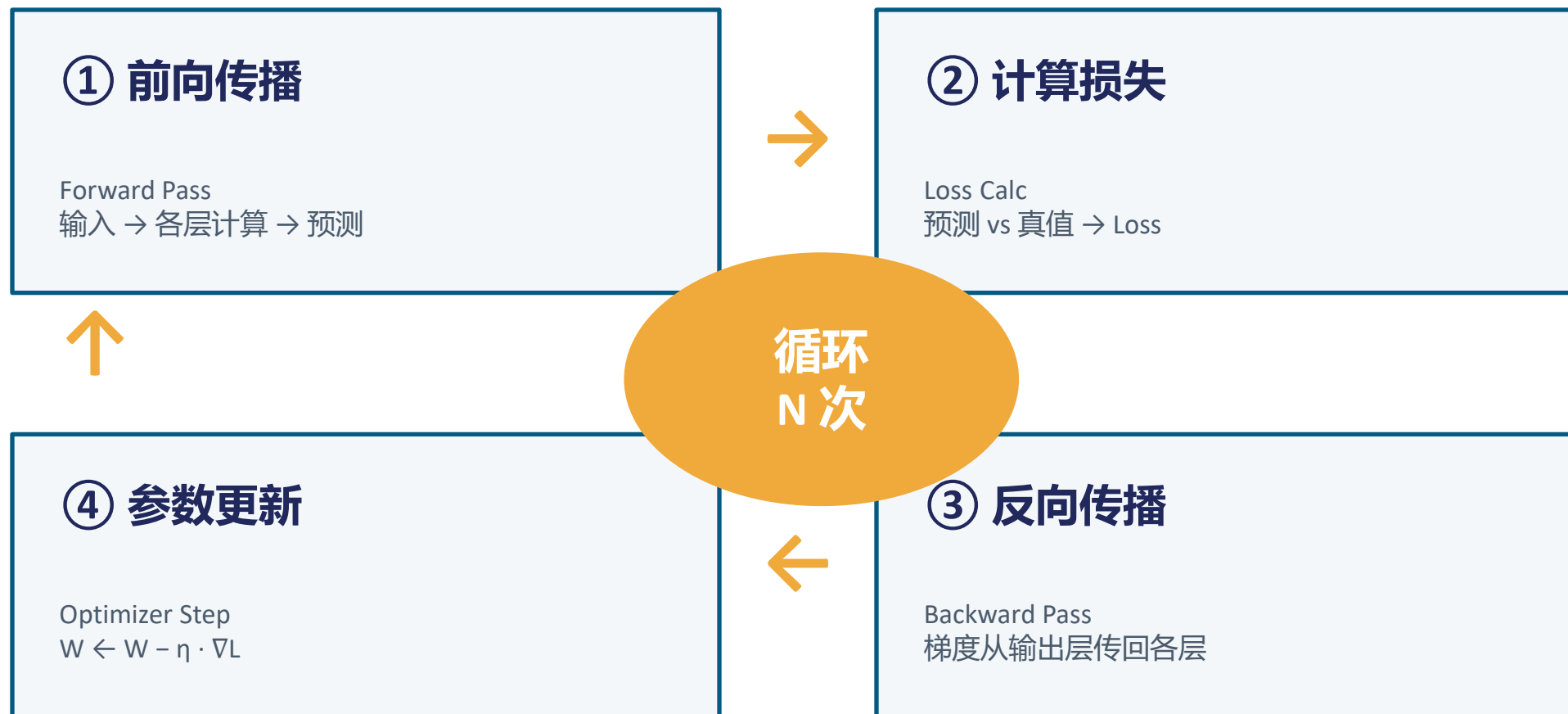
CHAPTER

深度学习与 Transformer

微调与量化的统一地基

智泊AI
ZHIPU AI魔泊云
MoPaaS

神经网络训练核心循环



LLM 微调常用 Epoch = 1-3 · Batch 通常 4-16 · LR $1e-4 \sim 5e-5$ (LoRA 通常 $1e-4$)

Transformer 整体架构

智泊AI
ZHIPO AI魔泊云
MoPaaS

Encoder · 编码器

理解输入文本

输入 Tokens → Embedding

+ Positional Encoding

Multi-Head Self-Attention

Add & LayerNorm

Feed-Forward Network

Add & LayerNorm

→ 输出隐状态

Decoder · 解码器

生成目标文本(GPT/LLaMA 类只用此塔)

起始符 → Embedding

+ Positional Encoding

Masked Self-Attention

(可选) Cross-Attention

Feed-Forward Network

Add & LayerNorm × N 层

→ Softmax → 下一 Token

★ GPT / LLaMA / Qwen 都是 **Decoder-only 架构**,通过 Next-Token Prediction 自回归生成

自注意力机制 · 核心公式

智泊AI
ZHIPU AI魔泊云
MoPaaS

$$Attention(Q, K, V) = softmax(QK^T / \sqrt{d_k}) \cdot V$$

1

生成 Q/K/V

输入 × 三个投影矩阵

2

计算 QK^T

Q 与所有 K 做点积

3

缩放 / $\sqrt{d_k}$

防止 softmax 饱和

4

Softmax

归一化为注意力权重

5

加权 V

权重 × V → 输出



Multi-Head Attention

用 h 个独立的注意力头,从不同子空间学习信息,再拼接融合 —— 一次看多个角度

例: GPT-2 small 有 12 个头 · Qwen2.5-7B 有 28 个注意力头

智泊AI
ZHIPU AI魔泊云
MoPaaS

Transformer 中的关键矩阵(为 LoRA 选层埋伏笔)

Self-Attention 模块

W_q (q_proj)

Query 投影矩阵

✓ 必选

W_k (k_proj)

Key 投影矩阵

✓ 必选

W_v (v_proj)

Value 投影矩阵

✓ 必选

W_o (o_proj)

输出投影矩阵

○ 可选

Feed-Forward 模块

gate_proj

门控线性层(LLaMA 系)

○ 可选

up_proj

升维线性层

○ 可选

down_proj

降维线性层

○ 可选

注: 门控/升降维矩阵在大型 MoE/LLaMA 模型上 LoRA 也常用, 但参数量更大

LoRA 默认 target_modules = ["q_proj", "k_proj", "v_proj", "o_proj"] ← 第4章会展开



智泊AI
ZHIPO AI



魔泊云
MoPaaS

03

CHAPTER

数据工程

微调成败的上游

微调数据 vs 预训练数据



维度	预训练数据	微调数据
规模	数百 GB ~ 数 TB	数百 ~ 数十万条
格式	原始文本流	结构化:指令-回答对
质量要求	数量优先	质量 >> 数量
标注	自监督,无需人工	人工 + LLM 合成
目标	学语言通用规律	学任务特定行为
成本	千卡 × 数月	单卡 × 数小时



一句话总结

预训练:
教模型「说人话」

微调:
教模型「该说什么」

数据质量,
决定上限

模型选择,
决定下限

三种数据格式速查

智泊AI
ZHIPU AI魔泊云
MoPaaS

SFT 数据

监督微调 · 最常用

```
{
  "instruction": "翻译为英文",
  "input": "今天天气真好",
  "output": "What a nice day!"
}
```

用途

指令理解 / 任务执行

偏好数据

DPO / RLHF

```
{
  "prompt": "推荐一本书",
  "chosen": "《人类简史》...",
  "rejected": "随便看吧"
}
```

用途

对齐人类偏好

工具调用

Function Calling

```
{
  "input": "查询北京天气",
  "tool_call": {
    "name": "get_weather",
    "args": {"city": "北京"}
  }
}
```

用途

Agent / 工具使用

智泊AI
ZHIPU AI魔泊云
MoPaaS

数据去重 · 三层次递进

L1

精确去重

技术 MD5 / SHA 哈希

示例 「今天天气真好」 ≠ 「今天 天气真好」

Exact Match



局限

改一个标点就漏

L2

近似去重

技术 SimHash / LSH / Jaccard

示例 相似 Token 模式可识别

Near-Duplicate



局限

改写为同义句仍漏

L3

语义去重

技术 Embedding + ANN + 阈值

示例 意义相近即识别(同义改写也能抓)

Semantic



局限

成本最高,需向量库

数据质量检测 · 六大维度



1

格式校验

字段完整性 / 编码 / 转义

方法 / 工具

正则 / Schema 校验

2

Perplexity

语言模型困惑度, 流畅度

方法 / 工具

PPL > 阈值 → 剔除

3

句法检查

语法是否成立

方法 / 工具

依存分析 / LLM 评估

4

术语一致性

专业表达统一

方法 / 工具

领域词典 + 规则替换

5

知识正确性

事实是否正确(最关键)

方法 / 工具

LLM-as-Judge + RAG 校验

6

标注一致性

结构与标注规范

方法 / 工具

三类对比 + 抽样审核

数据多样性 & Evol-Instruct 指令进化

智泊AI
ZHIPU AI魔泊云
MoPaaS

数据多样性 · 四维度

任务类型覆盖

翻译 / 摘要 / 分类 / QA / 生成均衡分布,防灾难性遗忘

难度分布

简单 / 中等 / 困难 合理混合

长度分布

短回答 + 中回答 + 长回答 均有

语言风格

正式 / 口语 / 技术,符合目标场景

Evol-Instruct · 指令进化

种子指令: 写一首关于秋天的诗



① 深度进化: 加约束(七言绝句)

② 推理进化: 加推理步骤要求

③ 复杂化: 增加输入复杂度

④ 增加难度: 多语言/多角色

⑤ 创造新任务: 迁移到不同领域

产物: 从 1 条种子 → 数十条多样化指令

特定场景的数据策略对照



场景	数据格式	规模参考	关键点
领域适配	SFT 单/多轮	1k – 50k	保持通用能力,避免灾难性遗忘
偏好对齐 DPO	(prompt, chosen, rejected)	5k – 50k	chosen/rejected 区分度要明显
Agent / 工具	含 tool_call 结构	1k – 10k	工具描述 + 参数 + 调用序列
多轮对话	messages 数组	5k – 50k	上下文衔接 + 角色一致
长文本	长文档 + 摘要/QA	1k – 5k 但长度大	确保真用到长上下文

★ 通用原则: 微调数据 质量 > 数量 ;1k 条高质量 > 10k 条噪声;领域微调需混 20%~30% 通用数据避免遗忘

★ 易错点 & 高频考点

智泊AI
ZHIPU AI魔泊云
MoPaaS

! 第3章 复习提示

X

数据越多越好

✓

微调数据质量 >> 数量。1k 高质量 SFT 比 10k 噪声更有效

X

只用领域数据微调

✓

会导致灾难性遗忘,需保留 20-30% 通用数据

X

精确去重就够了

✓

改一个字、换同义词都会绕过,需要近似 + 语义去重三层联防

X

对话模板可以省略

✓

训练与推理模板必须严格一致,否则效果断崖式下跌

X

长文本数据就是把短问题拼长

✓

必须是回答真正利用了长上下文信息,不然是浪费长度



智泊AI
ZHIPU AI



魔泊云
MoPaaS

04

CHAPTER

微调技术

从 LoRA 原理到 SWIFT 实战

★ 里程碑 M1

微调方法全景对比

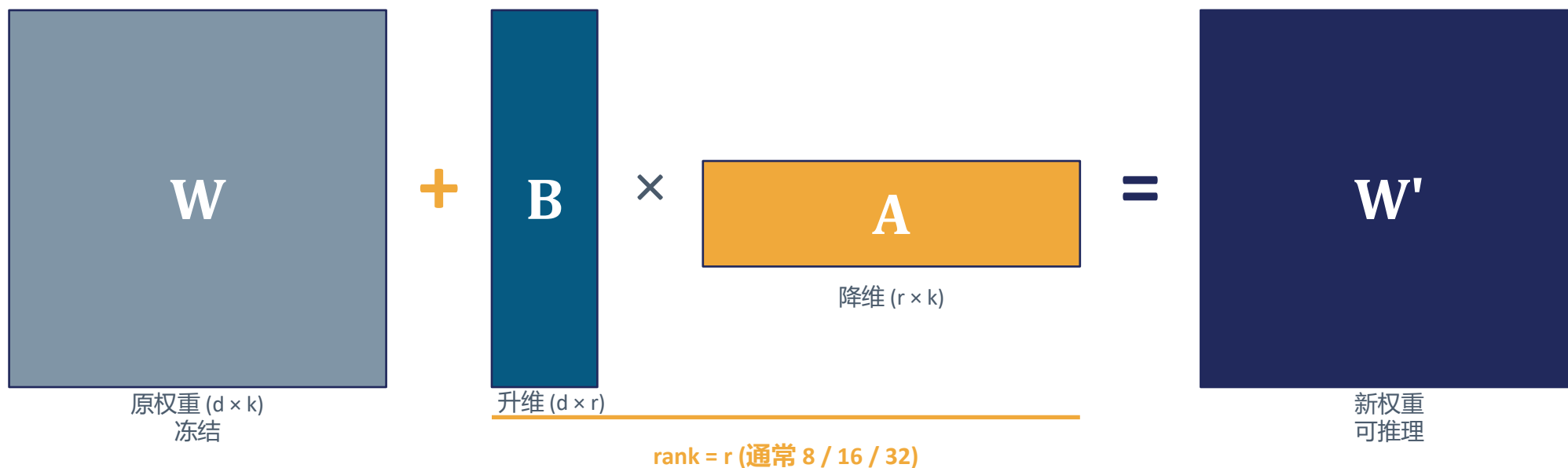


全参微调 <i>Full Fine-tuning</i>		LoRA <i>Low-Rank Adaptation</i>		QLoRA <i>Quantized LoRA</i>	
可训参数	100%	可训参数	0.1-1%	可训参数	0.1-1%
显存占用	极高	显存占用	中等	显存占用	极低
微调效果	★★★★★	微调效果	★★★★★	微调效果	★★★
训练成本	★★★★★	训练成本	★★	训练成本	★
适用场景		适用场景		适用场景	
大厂 / 充足算力 / 极致效果		★ 主流选择,效果 / 成本最优		单卡 24G 跑 70B,资源受限	

智泊AI
ZHIPU AI魔泊云
MoPaaS

LoRA 原理 · 低秩近似

$$W' = W + \Delta W = W + B \cdot A \quad (\text{其中 } \text{rank}(BA) = r \ll \min(d, k))$$



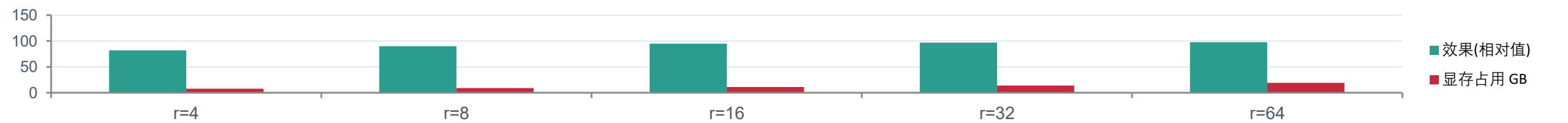
核心思想 W 冻结不训练,只学一对小矩阵 B/A 。可训参数从 $d \times k$ 降到 $r \times (d+k)$,通常缩小 100-1000 倍。**推理时可合并** $W' = W + BA$,无额外延迟。

LoRA 核心参数速查表



参数	作用	典型值	调整方向
r (秩)	低秩矩阵的秩,控制可训参数量	8 / 16 / 32 / 64	↑ 表达力↑,显存↑;数据少→小 r
lora_alpha	缩放因子,等价于学习率放大	16 / 32(常 = 2r)	通常固定为 2r,与 r 联动
lora_dropout	防止过拟合的丢弃率	0.05 – 0.1	数据少→ 0.1,数据多→ 0.05
target_modules	对哪些层应用 LoRA	[q,k,v,o]_proj	效果不够→ 加 FFN 层
bias	是否训练 bias 项	"none"	通常不训,保持 none

不同 r 值效果对比(显存与效果的权衡)



训练超参数 · 三大类速查

智泊AI
ZHIPU AI魔泊云
MoPaaS

数据 & 输入

决定输入数据形态

batch_size**4 – 16** 单步样本数**max_length****2048 – 8192** 最大序列长度**grad_accum_steps****4 – 16** 梯度累积,等效大 batch

优化

决定如何学习

learning_rate**1e-4 (LoRA)** 学习率,LoRA 比全参大**lr_scheduler****cosine** 学习率调度策略**warmup_ratio****0.03 – 0.1** 预热步数比例**num_epochs****1 – 3** 训练轮次

正则化 & 稳定性

决定训练稳健

weight_decay**0.01** 权重衰减,防过拟合**max_grad_norm****1.0** 梯度裁剪**bf16 / fp16****true** 混合精度训练

智泊AI
ZHIPU AI魔泊云
MoPaaS

Hugging Face 微调 · 七步流程

1

环境安装

`pip install transformers peft trl`

2

加载模型 / Tokenizer

`AutoModelForCausalLM.from_pretrained()`

3

数据集构建

`load_dataset()` + 格式化 + Chat Template

4

配置 LoRA

`LoraConfig(r=16, alpha=32, target_modules=...)`

5

训练参数

`TrainingArguments(lr, epochs, batch, ...)`

6

启动 SFT

`trainer = SFTTrainer(...) → trainer.train()`

7

合并权重

`model.merge_and_unload()` → 推理部署前

智泊AI
ZHIPU AI魔泊云
MoPaaS

SWIFT 三种使用方式 · 选哪个?



CLI

命令行

适用

脚本化 / 批量任务

```
swift sft \  
  --model qwen2-7b \  
  --dataset xxx.jsonl \  
  --use_lora true
```



Web UI

可视化

适用

★ 新手 / 探索式

```
swift web-ui  
# 浏览器打开  
# 点选 + 参数表单  
# 实时监控
```



Python API

代码调用

适用

集成 / 自动化流程

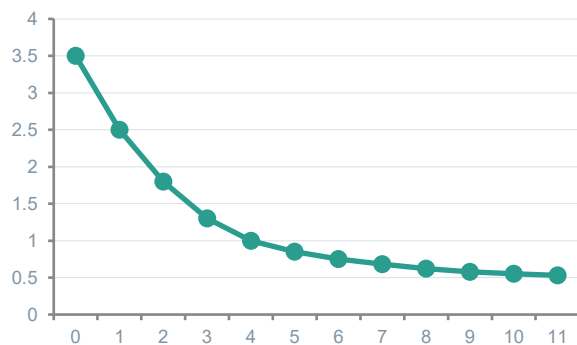
```
from swift.llm import \  
    sft_main, SftArguments  
sft_main(SftArguments(...))
```

智泊AI
ZHIPU AI魔泊云
MoPaaS

损失曲线 · 四种典型形态诊断

① 健康曲线

正常



现象

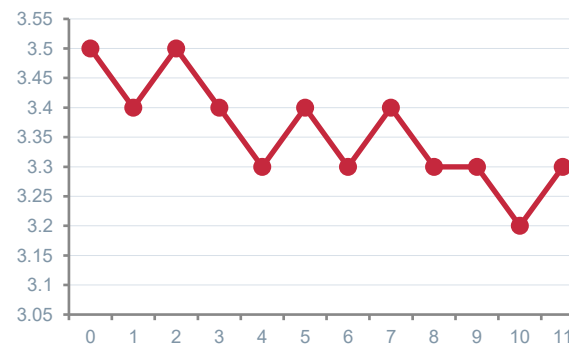
Loss 平稳下降并收敛

处理

继续训练或停止 ✓

② Loss 不下降

异常



现象

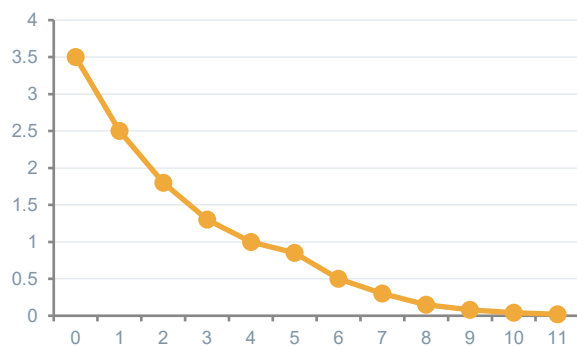
前期一直平,无法学习

处理

↑学习率 / 检查数据 / 换模板

③ 过拟合

警告



现象

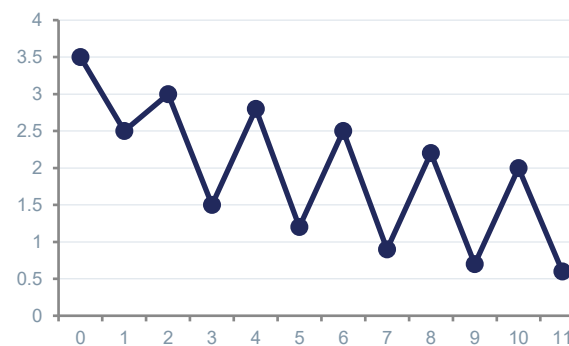
Loss 持续下降到极低,验证 Loss 反弹

处理

↓epochs / ↑dropout / ↑数据

④ 训练震荡

警告



现象

曲线大幅波动,不稳定

处理

↓学习率 / ↑grad_accum / 加 warmup

智泊AI
ZHIPU AI魔泊云
MoPaaS

微调评估 · 四类指标用法

1

词面重合

Lexical Overlap

字词层面的重合度

代表

BLEU · ROUGE · METEOR

适用

翻译 / 摘要 / 有标准答案

2

语义相似度

Semantic Similarity

向量空间的语义距离

代表

BERTScore · Sentence-BERT · Cosine Sim

适用

开放式问答 / 改写

3

分类 / 问答

Classification & QA

答案对错直接比对

代表

Accuracy · F1 · EM (Exact Match)

适用

分类 / 抽取式 QA / 选择题

4

事实性与一致性

Factuality

检测幻觉,忠实于原文/事实

代表

FactScore · LLM-as-Judge · QAGS

适用

RAG / 摘要 / 知识问答

★ 实用建议: 四类互补而非互斥,成熟方案 = 多指标组合 + 人工抽检 + 业务回归测试

★ 易错点 & M1 里程碑检查清单

! 易错点

X LoRA 的 lr 通常比全参微调大 $10\times$

X `target_modules` 默认只覆盖 `attention`, `FFN` 不动

X 推理前必须 `merge_and_unload()` 合并权重

X 训练与推理 Chat Template 必须严格一致

X 评估只看 Loss 不够, 要看业务指标



智泊AI
ZHIPU AI



魔泊云
MoPaaS



★ M1 里程碑

微调模型 + 评估报告

检查清单



数据集已清洗去重, 格式校验通过



选定模型 + LoRA 参数 + 合理 r 值



训练损失曲线健康收敛



已合并权重并可独立推理



至少 2 类指标 + 人工抽检



对比基座模型有明显提升



覆盖业务测试 + RAG + 拒答



智泊AI
ZHIPU AI



魔泊云
MoPaaS

05

CHAPTER

模型量化与压缩

让模型瘦身又能跑

★ 里程碑 M2

智泊AI
ZHIPU AI魔泊云
MoPaaS

为什么需要量化 · 三大痛点

1

模型太大

Qwen2.5-7B FP16 = 14 GB

笔记本 / 消费级显卡跑不动

→ 量化到 INT4, 只需 4 GB

2

推理太慢

Token/s 下降 + 显存带宽瓶颈

用户等待时间长, 体验差

→ INT8 / INT4 提速 2-4×

3

部署太贵

高端 GPU 单卡 \$5万+

全量精度需要 80GB 显存的 A100

→ 量化后 24GB 4090 即可

数据类型全谱速查



类型	位宽	范围 / 精度	典型用途	7B 模型大小
FP32	32 bit	全精度浮点	训练 / 科学计算	28 GB
FP16	16 bit	半精度,范围窄易溢出	训练 + 推理	14 GB
BF16	16 bit	范围宽,精度低	★ LLM 训练首选	14 GB
INT8	8 bit	-128 ~ 127 均匀整数	推理量化	7 GB
INT4	4 bit	-8 ~ 7 均匀整数	极致压缩推理	3.5 GB
NF4	4 bit	★ 16 个非线性浮点刻度	QLoRA 微调专用	3.5 GB



NF4 (Normal Float 4-bit) · 为什么特殊?

- ▶ 不是均匀整数,而是 16 个非线性浮点刻度,密度匹配 **正态分布**(神经网络权重的真实分布)
- ▶ 中心密、两端疏,精度损失最小化
- ▶ QLoRA 的核心数据类型,把 4-bit 量化的精度损失降到接近 FP16

智泊AI
ZHIPU AI魔泊云
MoPaaS

PTQ 双雄 · GPTQ vs AWQ 设计哲学

GPTQ

基于梯度的训练后量化

设计哲学

事后修正

核心关键词

误差补偿

原理

逐列量化,把误差转嫁给未处理的列

流程

分组量化 → Hessian 估计影响 → 三步算法补偿

通用场景,稳定可靠

AWQ

激活感知量化

设计哲学

事前保护

核心关键词

重要权重保护

原理

只有 1% 权重重要,保护它们不被量化

流程

激活值统计 → 识别重要 channel → 缩放保护

★ 推理服务,vLLM 主推

智泊AI
ZHIPU AI魔泊云
MoPaaS

QLoRA · 量化 + 低秩微调的四大机制

1

NF4 权重量化

Normal Float 4-bit

16 个非线性浮点刻度,匹配权重正态分布

2

双量化

Double Quantization

对量化常数本身再量化,平均节省 0.37 bit/参数

3

LoRA 低秩适配

Low-Rank Adapter

冻结的量化权重 + 可训练 LoRA 模块协同

4

分页优化器

Paged Optimizer

GPU 内存峰值溢出时,自动换页到 CPU 内存

★ 实战价值: 单张 24G 4090 微调 65B 模型,效果接近全参,显存降低 3-4×

GGUF 格式 · 本地部署的事实标准



格式 ≠ 算法 GGUF 是 **存储格式**,装的是各种量化算法(Q4_K_M, Q5_K_S 等)产生的权重

单文件部署

权重 + 元数据 + Tokenizer 全在一个 .gguf 文件

跨平台运行

llama.cpp + Ollama + LM Studio,Mac / Linux / Windows

灵活量化

K-Quant 自适应分组,精度 / 大小灵活权衡

量化等级选择对照(以 7B 模型为例)

等级	位宽	大小	质量	适用
Q8_0	8 bit	7.2 GB	★★★★★	近 FP16,质量优先
Q5_K_M	5 bit	4.8 GB	★★★★★	★ 推荐折中
Q4_K_M	4 bit	4.1 GB	★★★★★	★★ 最常用
Q3_K_S	3 bit	3.0 GB	★★	极限压缩

模型蒸馏 · 把大模型的「暗知识」装进小模型

智泊AI
ZHIPU AI魔泊云
MoPaaS

Dark Knowledge · 暗知识 大模型输出的**概率分布**,本身就携带比硬标签更多的信息(类间相似度等)

三大流派演进

① 输出对齐

Logit Distillation
软标签 + 硬标签联合

② 中间层对齐

Feature Distillation
隐藏层 / 注意力图对齐

③ 关系结构对齐

Relation Distillation
样本间关系矩阵对齐

LLM 时代的蒸馏(2023-2025)

- ▶ **数据蒸馏** 用大模型生成 SFT 数据训练小模型(Alpaca、Vicuna 路线)
- ▶ **推理能力蒸馏** 蒸馏 CoT 思维链(o1 → o1-mini, DeepSeek-R1-Distill)

蒸馏 vs 量化 vs 剪枝

蒸馏改架构 · 量化改精度 · 剪枝改稀疏 · 三者可组合使用

★ 易错点 & M2 里程碑检查清单



易错点

X 量化 ≠ 直接降参,需要标定数据集

X 性能损失必须实测,不能只看大小

X AWQ 推理快但量化慢;GPTQ 反之

X QLoRA 是微调不是部署方案

X GGUF 主用 llama.cpp 路线,生产 GPU 服务用 AWQ + vLLM



★ M2 里程碑

量化模型 + 性能对比报告

检查清单

- ✓ 选定量化方案 (AWQ / GPTQ / GGUF)
- ✓ 完成量化并验证模型可加载
- ✓ 性能指标对比(吞吐/延迟/显存)
- ✓ 效果指标对比(原 vs 量化)
- ✓ 可接受的精度损失说明
- ✓ 压缩率与加速比汇总



智泊AI
ZHIPU AI



魔泊云
MoPaaS

06

CHAPTER

推理引擎部署

从模型到服务的最后一公里

★ 里程碑 M3

推理服务 · 五大核心指标

智泊AI
ZHIPU AI魔泊云
MoPaaS

1	2	3	4	5
吞吐量 <i>Throughput</i>	延迟 <i>Latency</i>	并发数 <i>Concurrency</i>	显存 <i>VRAM</i>	成本 <i>Cost</i>
tokens/s	TTFT / TPOT	req/s	GB	\$/M token
单位时间产生的 Token 数	首 Token 时间 / 每 Token 时间	并行处理的请求数	推理占用的 GPU 内存	每百万 Token 的总成本

核心张力 ↑ 吞吐(批大)往往导致 ↑ 延迟,部署需找平衡点 · 见 P47 Rooftop 曲线

显存预算速查公式

智泊AI
ZHIPU AI魔泊云
MoPaaS

$$VRAM \approx \text{模型权重} + KV\ Cache + \text{激活} + \text{框架开销}$$

模型权重

$$\text{参数量} \times \text{位宽} / 8$$

7B FP16 = 14 GB
7B INT4 = 3.5 GB

KV Cache

$$2 \times \text{层数} \times \text{头数} \times \text{seq_len} \times \text{头维} \times \text{位宽} / 8 \times \text{batch}$$

通常 10–40% 权重
占总显存 1/3 ~ 1/2

激活

$$\text{batch} \times \text{seq_len} \times \text{hidden} \times \text{系数}$$

短上下文小,长上下文显著

框架开销

1–2 GB CUDA Context + 库

PyTorch 启动占用
基本固定

算例: RTX 4090 (24GB) 跑 Qwen2.5-7B AWQ-INT4, 4k 上下文, batch=4

权重: $7B \times 0.5 \text{ byte} = 3.5 \text{ GB}$ + **KV Cache:** $2 \times 28 \times 28 \times 4096 \times 128 \times 2 \text{ byte} \times 4 \approx 5.4 \text{ GB}$
+ **激活:** 约 1.5 GB + **框架:** 约 1.5 GB $\approx$ **总计 ~12 GB** (24GB 卡剩 12GB 余量, 可加大 batch 或 seq)

三大推理优化技术

智泊AI
ZHIPU AI魔泊云
MoPaaS

Continuous Batching

连续批处理

问题

传统批处理需等齐才返回,资源浪费

解决

每步动态加入新请求,完成的立刻释放

吞吐 ↑ 2-5×



Paged Attention

vLLM 核心创新

问题

KV Cache 静态预分配,大量碎片浪费

解决

分页式 KV Cache,按需分配 + 复用

显存利用率 90%+



Prefix Caching

前缀缓存

问题

相同 System Prompt 反复计算

解决

共享前缀的 KV 直接复用,跳过 prefill

TTFT ↓ 50-90%

引擎选型 · vLLM vs SGLang

智泊AI
ZHIPU AI魔泊云
MoPaaS

vLLM

UC Berkeley · 通用GPU 事实标准

关键特性

- ✓ 成熟稳定,生态最广
- ✓ OpenAI 兼容 API
- ✓ PagedAttention 原创
- ✓ 支持 AWQ/GPTQ 量化
- ✓ 持续支持新模型

★ 通用聊天 / RAG / 单轮生成 / 生产首选

SGLang

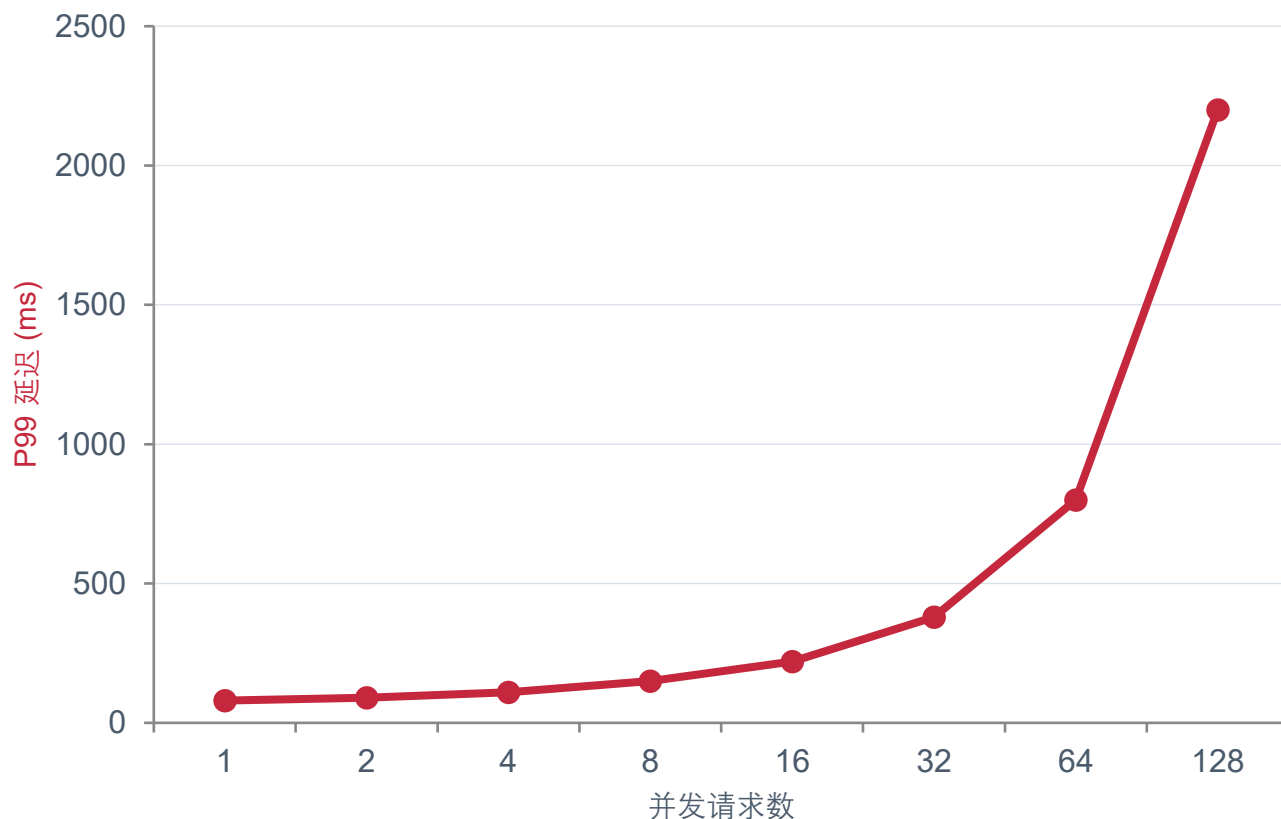
LMSYS · Agent 与长上下文新王

关键特性

- ✓ RadixAttention(更强前缀复用)
- ✓ 结构化输出原生支持
- ✓ ★ 多轮 Agent 性能更优
- ✓ 长上下文 prefix 命中率高
- ✓ FlashInfer 集成

★ Agent 系统 / 长会话 / 结构化输出 / 复杂分支

吞吐 vs 延迟 · 核心张力

智泊AI
ZHIPU AI魔泊云
MoPaaS

Rooftop 曲线读法

✓ 安全区

并发 ≤ 16 , 延迟平稳

△ 拐点

并发 16-32, 延迟开始上扬

✗ 崩溃区

并发 > 64 , 延迟雪崩, KV 显存打满

找到拐点

生产部署应在 SLO 容许的最大并发上, 即拐点附近运行, 既保延迟又保吞吐

★ 部署 SOP: ① 用真实业务流量压测 → ② 绘制吞吐/延迟曲线 → ③ 找到 SLO 边界 → ④ 留 30% 余量做容量规划

压测的五大陷阱



智泊AI
ZHIPU AI



魔泊云
MoPaaS

1

只看吞吐不看延迟

吞吐高, P99 延迟可能爆炸; 必须同时盯 TTFT / TPOT / P99

2

短 prompt 短 output

测试 prompt 长度与生产差距大, KV 行为完全不同

3

并发数 = 用户数

用户有 think time, 1000 用户 ≠ 1000 并发, 见 P50

4

不预热

首次请求要加载模型 + 编译 kernel, 前几秒数据无效

5

单一 prompt 重复发

Prefix Caching 让数据虚高, 真实场景会大幅下降

GPU 选型速查



GPU	显存	带宽	FP16 算力	\$ 区间	适用场景
RTX 4090	24 GB	1 TB/s	165 TFLOPS	万元级	★ 个人 / 7B-14B 量化推理
L40S	48 GB	0.86 TB/s	362 TFLOPS	5万级	中型推理服务,性价比高
A100	40/80 GB	1.5-2 TB/s	312 TFLOPS	10万级	训练 + 推理,经典选择
H100 / H200	80/141 GB	3.4-4.8 TB/s	989 TFLOPS	30万级	★ 训练大模型 / 极致性能
国产 (910B 等)	32-64 GB	—	—	—	信创合规场景

四大反直觉认知

◆ 推理瓶颈通常是带宽不是算力

◆ 多卡 ≠ 线性加速,通信开销可能吃掉收益

◆ 更便宜的卡 × N 不一定比 A100 单卡省

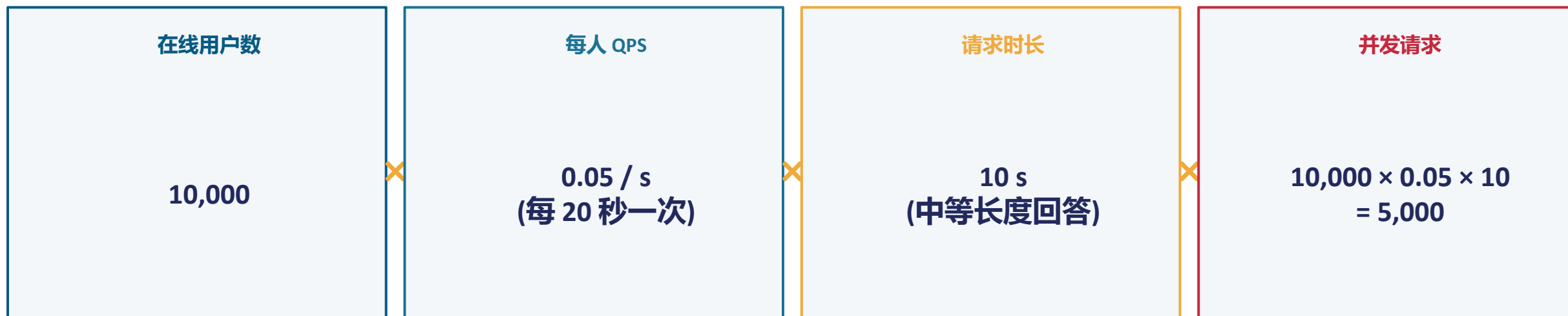
◆ 显存够用就行,过度配置不划算

并发数 ≠ 用户数 · 容量换算

智泊 AI
ZHIPU AI魔泊云
MoPaaS

并发请求数 $\approx$ 在线用户数 $\times$ QPS/用户 $\times$ 平均请求时长

实例换算(智能客服场景)



结论 1 万在线用户 $\neq$ 1 万并发,实际只需支撑 **5000 并发**;按拐点 32 并发/实例算,需 **~150 个 vLLM 实例**(留 30% 余量)

★ 忽略 think time 直接按用户数算,会过度配置 3-5 倍硬件

★ M3 里程碑检查清单 + 部署七句话

部署速记七句话

- 1 量化模型选 AWQ INT4(vLLM 友好)
- 2 推理引擎选 vLLM(通用)或 SGLang(Agent)
- 3 OpenAI 兼容 API,便于客户端切换
- 4 容器化部署 + 环境变量配置
- 5 压测:真实流量 + 完整指标 + 留 30% 余量
- 6 监控:GPU 利用率 + QPS + 延迟 + 错误率
- 7 上线后小流量灰度,逐步扩容

智泊 AI
ZHIPU AI魔泊云
MoPaaS

★ M3 里程碑

服务可被调用 + Docker 镜像

检查清单

- ✓ vLLM / SGLang 服务正常启动
- ✓ OpenAI 兼容 API 可调用
- ✓ 完整压测报告(吞吐 + 延迟)
- ✓ 容量规划(并发上限)
- ✓ GPU 选型论证
- ✓ Docker 镜像 + 启动脚本
- ✓ 健康检查 + 监控埋点



智泊AI
ZHIPO AI



魔泊云
MoPaaS

07

CHAPTER

RAG 文档处理

让模型读懂私有知识

文档处理 · RAG 的入口质量门

智泊AI
ZHIPU AI魔泊云
MoPaaS

典型失败案例

某金融客户上线 RAG 系统,准确率持续低于 60%。追根溯源——PDF 解析时,关键表格被拆成乱序文字,数字错位。模型本身没问题,文档处理一开始就错了。

RAG 完整链路 · 每一环都可能放大错误



文档处理错 5%,经过分块/检索两步放大,最终错误率可能 ↑ 到 30%+

★ **经验法则**: 投入 60% 精力在文档解析与分块,40% 在后续检索生成,而不是反过来

- ▶ 电子文档要解结构(标题、列表、表格)
- ▶ 扫描件要选好 OCR 引擎(PaddleOCR / MinerU)
- ▶ 高价值且形态复杂的文档,考虑视觉原生方案(ColPali)

五类文档形态对照



1	电子文档 <i>.docx .pdf .md</i>	原生数字文档,可直接解析	pdfplumber / python-docx / Unstructured
2	扫描件 <i>PDF 截图 / 图片</i>	本质是图像,需 OCR	PaddleOCR / MinerU / Tesseract
3	表格 <i>财务报表 / 合同</i>	需保持行列结构	Camelot / Tabula / PaddleOCR-Struct
4	公式 / 图表 <i>论文 / 教材</i>	数学符号 + 含义提取	Nougat / Mathpix / MinerU
5	工程图 <i>电路 / 建筑图纸</i>	图形 + 标注,极难	视觉原生 / 多模态大模型

三大技术范式对比 · 哪种适合你?



传统 OCR <i>Pipeline 工程</i>	电子文档解析 <i>原生结构提取</i>	视觉原生 RAG <i>End-to-End 多模态</i>
流程 图像 → OCR → 后处理 → 文本 优势 ✓ 生态成熟 ✓ 可控性强 ✓ 对扫描件最优 局限 X 流程复杂 X 错误累积 X 复杂版式难 扫描件主导场景	流程 DOC/PDF → 解析器 → 结构化文本 优势 ✓ 速度最快 ✓ 结构最完整 ✓ 适合电子原稿 局限 X 仅适用电子原生 X 扫描件无能为力 ★ 90% 内部知识库	流程 图像 patch → 多向量 → late-interaction 优势 ✓ 保留版面信息 ✓ 对复杂图表强 ✓ 无 OCR 中间损失 局限 X 计算成本高 X 存储 5-10× X 模型门槛 高价值复杂文档 / 视觉信息关键

主流工具链全景图



视觉原生 RAG · ColPali 原理

智泊 AI
ZHIPU AI魔泊云
MoPaaS

核心思想 不走 OCR, 直接对 **图像 Patch** 做 embedding · 查询时通过 **late-interaction** 细粒度匹配

索引阶段(离线)



查询阶段(在线)



三大代价(成本权衡)

存储成本 5-10×

每页 1000+ patch 向量, 远超传统单向量

计算开销大

late-interaction 不可预聚合, 查询需 $O(N \times M)$ 矩阵运算

模型门槛高

需要多模态 VL 模型 + GPU 推理基础设施

分块策略 × 文档类型映射



策略	原理	优势	适用文档类型
固定切分	按 token 或字符切分	实现最简单,速度快	结构松散文本,日志
递归切分	按段落 → 句 → 词层级递归	★ 通用且尊重边界	一般纯文本(LangChain 默认)
语义切分	用 embedding 检测语义边界	块内主题更聚焦	话题切换频繁的文档
文档结构切分	按标题/小节/列表/表格	★ 保留语义结构	技术文档 / 知识库 / 教材
视觉感知切分	保留版面 + 表格 + 图	对复杂版面最优	PDF 报告 / 论文 / 财报

★ 实战建议

- 块大小:通常 300-800 token,过小信息不全,过大稀释相关性
- 重叠:10-20% 重叠,缓解边界信息切断;过多则冗余
- 结构优先:技术文档 / 规范类,优先文档结构切分,保留小节标题作为元数据

选型决策 · 双维度四象限



国产工业级 推荐组合

- 解析
- Unstructured + MinerU
- Embedding
- BGE-M3 / BGE-large-zh
- 向量库
- Milvus / Qdrant
- 重排
- BGE-Reranker / Cohere
- 复杂文档
- ColQwen2 (视觉原生)
- 生成
- Qwen2.5 + vLLM

RAG 评估 · 三层体系



★ 易错点 & 高频考点

智泊AI
ZHIPU AI魔泊云
MoPaaS

! 第7章 复习提示

X

RAG 效果不好,先换模型

✓

90% 问题在文档处理 + 分块,而不是模型

X

所有文档都用同一种分块

✓

技术文档按结构、报告按视觉感知、纯文本用递归切分

X

embedding 选维度越高越好

✓

维度越高存储越大,BGE-M3 1024 维已经够用

X

用了知识图谱就高级

✓

构建成本极高,绝大多数场景过度工程

X

不评估就直接优化

✓

必须先建黄金集 + 三层评估,定位瓶颈再优化



智泊AI
ZHIPU AI



魔泊云
MoPaaS

08

CHAPTER

智能体新生态

从调用模型到模型驱动 workflow

智能体 vs Prompt 工程 vs 微调 · 三者定位



L0 Prompt 工程	L1 智能体	L2 微调
是什么 改提示词 适用 通用任务,模型本身够强 成本 几乎为零 典型示例 Few-shot + CoT + 模板	是什么 Harness Engineering 适用 ★ 任务复杂、需多步推理或调工具 成本 中等(开发 + 调试) 典型示例 Skill + MCP + workflow	是什么 改模型参数 适用 特定领域知识 / 风格 / 任务格式 成本 高(数据 + 训练) 典型示例 SFT / LoRA / DPO

OpenClaw · 智能体平台架构

智泊AI
ZHIPU AI魔泊云
MoPaaS

用户接入层

Web UI

API

命令行 CLI

智能体核心层

对话管理

workflow引擎

上下文管理

决策规划

能力扩展层

Skill 模块 (50+)

MCP Server 协议

工具调用

外部 LLM

★ **Harness Engineering** —— 不改模型,只 **设计模型的「使用框架」**。OpenClaw 提供框架,开发者填充业务

OpenClaw 工作原理 · 五步法



1	理解意图 解析用户输入,识别任务类型 + 实体 + 约束
2	规划任务 拆解为子任务,确定执行顺序与依赖关系
3	选择 Skill 匹配最合适的 Skill / MCP / 内部工具
4	执行调用 调用模型 / 外部 API,获取结果并回填上下文
5	整合反馈 聚合中间结果,生成最终回答并记忆要点

Skill 与 MCP · 能力扩展双引擎



Skill 开发

OpenClaw 原生模块

定义

封装单个能力的代码 + 提示词

适用

私有业务逻辑 / 紧密集成

形式

```
skill.json + handler.py
```

示例

财务报表分析 · 合同审核 · 工单处理

MCP 协议

Model Context Protocol

定义

标准化 LLM 与外部工具的接口

适用

★ 通用 / 跨厂商 / 复用社区

形式

```
MCP Server (HTTP / stdio)
```

示例

GitHub / Slack / Notion / DB 查询

协同模式

Hybrid Pattern

定义

私有能力用 Skill,外部接入用 MCP

适用

★ 实战推荐

形式

```
Skill + MCP 编排
```

示例

内部用 Skill 跑业务,MCP 接 Slack 通知

AI 辅助编程工具横向对比



工具	厂商	形态	亮点	适用
Cursor	Anysphere	IDE	全局上下文,深度集成	★ 全栈开发者
Claude Code	Anthropic	CLI	★ Agent 化命令行,长任务能力	命令行重度
Copilot	GitHub / MS	插件	生态最广,起步早	通用补全
CodeBuddy	腾讯	IDE 插件	国产合规,中文优化	国内企业
通义灵码	阿里云	IDE 插件	私有部署友好	云上开发



选型逻辑

- ▶ 数据敏感度高 / 国产化要求 → **CodeBuddy / 通义灵码**
- ▶ 追求极致体验 / 全栈开发 → **Cursor**
- ▶ 命令行深度用户 / Agent 化任务 → **Claude Code**
- ▶ 团队协作 / 已用 GitHub → **Copilot**



智泊AI
ZHIPU AI



魔泊云
MoPaaS

09

CHAPTER

综合实战

电商智能客服系统

★ 里程碑 M4

项目全貌 · 服务架构一图懂



一次完整用户请求路径

用户提问 → Gradio 收集 → FastAPI 路由 → Redis 取上下文 → RAG 检索知识 → 拼 Prompt → vLLM 生成 → 流式返回 → Redis 更新会话 → Gradio 展示

关键技术映射(贯穿全课程)



从零到生产 · 七步工程流程

**智泊AI**
ZHIPU AI**魔泊云**
MoPaaS

1	需求 + 数据准备	业务场景定义 + 客服对话语料收集 + 知识库整理	Ch1, 3
2	模型微调	Qwen1.5-1.8B-Chat + LoRA + 客服话术数据,SWIFT 训练	Ch4
3	模型量化	AWQ W4A16 量化,显存降至 ~2GB,准备 vLLM 部署	Ch5
4	推理服务	vLLM 启动 OpenAI 兼容 API,GPU 推理服务化	Ch6
5	RAG 集成	BGE-M3 索引知识库 + Milvus 向量库 + 检索拼接 Prompt	Ch7
6	服务架构	FastAPI 编排 + Redis 会话 + Gradio 前端 + Docker 部署	新增
7	测试与上线	13 项功能测试 + 压测 + 灰度发布	Ch6

关键技术决策汇总(贯穿全课程)



决策点	本项目选择	为什么	对应章节
基座模型	Qwen1.5-1.8B-Chat	中文场景强,体积小,可单卡部署	第 1 章
微调方法	LoRA + SWIFT	效果 / 成本最优,SWIFT 易用	第 4 章
量化方案	AWQ W4A16	推理快,vLLM 原生支持	第 5 章
推理引擎	vLLM	通用场景成熟标准	第 6 章
Embedding	BGE-M3	国产中文最强,多语言通用	第 7 章
向量库	Milvus	开源工业级,生态成熟	第 7 章
前端 / 服务	Gradio + FastAPI + Redis	快速验证 + 可扩展	本章

★ 这张表就是 M4 的核心 —— 完整跑通的电商客服系统全栈技术选型

每个决策都不是「最先进」,而是「在约束下最合适」,这才是工程师该有的判断力

系统架构 · 四层抽象



13 项核心功能 + GUI 测试用例

1 商品咨询

2 订单查询

3 退换货

4 物流追踪

5 支付问题

6 优惠政策

7 会员服务

8 投诉处理

9 技术故障

10 账户管理

11 上下文记忆

12 流式输出

13 敏感拦截

三区服务设计		
业务区(1-10)	能力区(11-12)	安全区(13)
电商客服核心场景,RAG 驱动,知识库支撑	上下文记忆 + 流式输出,提升用户体验	敏感词拦截 + 内容审核,合规底线

★ 部署要点 & M4 里程碑检查清单

部署要点



一键启动

docker-compose up, 一条命令拉起全栈



显存规划

Qwen-1.8B-AWQ ~2GB, 加上 KV Cache 总计 ~8GB



离线模型

模型预下载本地, 避免首次启动等待



环境变量

敏感配置(API Key、数据库密码)走 .env



健康检查

/health 接口 + 监控埋点 + 错误告警



★ M4 里程碑

完整可演示的智能客服系统

检查清单



微调 + 量化的模型已部署 (M1+M2)



vLLM 推理服务可调用 (M3)



RAG 知识库可检索可命中



Gradio 前端可访问可对话



FastAPI 网关 + Redis 上下文



13 项核心功能全部跑通



Docker 一键启动验证



压测与基础监控就位



四大里程碑全景回顾



M1 微调模型	交付 关键 章节	可工作的微调模型 + 评估报告 LoRA 配置 · SWIFT 训练 · 多指标评估 Ch3-4 · 数据 + 微调
M2 量化模型	交付 关键 章节	量化后模型 + 性能对比 AWQ W4A16 · 显存降低 4× · 精度可控 Ch5 · 量化与压缩
M3 服务可调用	交付 关键 章节	vLLM 服务 OpenAI 兼容 API · 容量规划 · 压测 Ch6 · 推理引擎部署
M4 完整系统	交付 关键 章节	电商客服全栈系统 Gradio + FastAPI + RAG + vLLM + Redis Ch7-9 · RAG + Agent + 实战

学习闭环 & 后续进阶方向

✓ 这门课带走了什么

✓ 从开源模型到生产服务的完整链路

✓ LoRA / QLoRA 微调实战能力

✓ 量化与推理优化的工程经验

✓ RAG 系统设计与文档处理

✓ Agent 平台架构与 Skill+MCP

✓ 用一个完整项目串联所有技术点

→ 还可以往哪里走

→ **强化学习对齐** DPO / PPO / GRPO,提升模型偏好

→ **多模态扩展** Vision-Language 模型 (Qwen-VL)

→ **长上下文优化** YaRN / 100K+ context 工程

→ **MoE 与大模型工程** DeepSeek-V3 风格的稀疏架构

→ **Agent 系统设计** Multi-agent / Planning / Reflection

→ **推理优化前沿** Speculative Decoding / MLA / RadixAttention





智泊AI
ZHIPU AI



魔泊云
MoPaaS

THANK YOU

感谢这门课让我们走完了从微调到部署的完整旅程

剩下的,就是动手把模型跑起来

✓ M1 微调

✓ M2 量化

✓ M3 部署

✓ M4 系统